STM32L452RE I²C Driver Development

A Bare-Metal Register-Level Lecture Series

Covering I²C Initialization, Timing Configuration,
Master Transmission, and Master Reception

Compiled Lecture Notes — Lectures 1 through 7

Table of Contents

LECTURE 1: Implementing I2C_Init() on STM32L452RE — Configuring CR1, TIMINGR, OAR1, and CR2

Objective

In this lecture, we begin implementing the I2C_Init() function for the STM32L452RE. Unlike older STM32 families, the STM32L4 series uses the second-generation I²C peripheral, which replaces the legacy CCR and TRISE registers with a single TIMINGR register.

By the end of this lecture, the driver will:

- Configure the I²C peripheral while it is disabled.
- Configure the Own Address Register (OAR1).
- Configure optional CR1 features.
- Configure timing using the TIMINGR register.
- Prepare the peripheral for communication.

STM32L452RE I²C Peripheral Overview

The STM32L452RE contains three I²C peripherals:

Peripheral	Bus
I2C1	APB1
I2C2	APB1
I2C3	APB1

All three peripherals are connected to the APB1 peripheral bus.

Before configuring any peripheral:

- Enable its peripheral clock.
- Configure the GPIO pins.
- Disable the I²C peripheral.
- Configure registers.
- Re-enable the peripheral.

Initialization Flow

```
I2C_Init()

      |
      ▼
Disable I2C Peripheral (PE = 0)

      |
      ▼
Configure CR1

      |
      ▼
Configure TIMINGR

      |
      ▼
Configure OAR1

      |
      ▼
Configure CR2 (optional defaults)

      |
      ▼
Enable Peripheral (PE = 1)
```

Unlike older STM32 devices, TIMINGR must be programmed while the peripheral is disabled.

Step 1 — Disable the Peripheral

The PE (Peripheral Enable) bit is located in CR1 Bit 0.

Register	Bit	Name
CR1	0	PE

Before changing timing parameters,

PE = 0

must be ensured.

```
pI2Cx->CR1 &= ~(1 << I2C_CR1_PE);
```

Why?

The Reference Manual specifies that timing registers cannot be modified while the peripheral is enabled.

Step 2 — Configure CR1 Register

Unlike the legacy peripheral, CR1 no longer contains ACK control.

Instead, CR1 enables optional features.

Common configuration bits include:

Bit	Field	Purpose
0	PE	Peripheral Enable
12	ANFOFF	Analog Filter Disable
8	TXIE	Transmit Interrupt
2	RXIE	Receive Interrupt
1	TXDMAEN	DMA Transmit
15	NOSTRETCH	Clock Stretch Disable

For a blocking driver, most interrupt and DMA bits remain disabled.

Recommended configuration:

- Analog filter enabled
- Clock stretching enabled
- Interrupts disabled
- DMA disabled

Initially:

```
uint32_t tempreg = 0;
```

If clock stretching must be disabled,

```
tempreg |= (1 << I2C_CR1_NOSTRETCH);
```

Finally,

```
pI2Cx->CR1 = tempreg;
```

Notice that PE is not enabled yet.

Step 3 — Configure Own Address Register (OAR1)

If the peripheral ever operates as a slave, it requires an address.

The register used is

OAR1

Important fields:

Bits	Purpose
10:0	Own Address
15	OA1EN

Example:

Suppose

Own Address = 0x52

Then

```
tempreg = 0;

tempreg |= (0x52 << 1);

tempreg |= (1 << 15);

pI2Cx->OAR1 = tempreg;
```

For pure master applications, configuring OAR1 is optional but recommended.

Step 4 — Understanding TIMINGR

The STM32L452RE replaces the old

CCR
TRISE

registers with

TIMINGR

This register completely determines the I²C timing.

Register layout

```
31                    0

+--------+--------+--------+--------+--------+
|PRESC|SCLDEL|SDADEL|SCLH|SCLL|
+--------+--------+--------+--------+--------+
```

Fields

Field	Purpose
PRESC	Clock prescaler
SCLDEL	Data setup delay
SDADEL	Data hold delay
SCLH	SCL high period
SCLL	SCL low period

Unlike older devices,

There is no CCR calculation.

There is no TRISE register.

All timing information is packed into TIMINGR.

Step 5 — Configuring TIMINGR

The timing value depends upon

- Peripheral clock frequency
- Desired I²C speed
- Rise time
- Fall time
- Analog filter
- Digital filter

ST provides timing values using

STM32CubeMX or the official I2C Timing Configuration Tool.

Example

Assume

PCLK1 = 80 MHz

I2C Speed = 100 kHz

A commonly used timing value is

TIMINGR = 0x10909CEC

Programming the register

```
pI2Cx->TIMINGR = 0x10909CEC;
```

Later in the course, we will generate timing values dynamically instead of hardcoding them.

Step 6 — Configure CR2 Default State

CR2 contains

- Slave address
- Number of bytes
- AUTOEND
- RELOAD
- START
- STOP

These fields are used during data transfers.

During initialization,

CR2 is cleared.

```
pI2Cx->CR2 = 0;
```

Step 7 — Enable the Peripheral

After every register has been configured,

enable the peripheral.

```
pI2Cx->CR1 |= (1 << I2C_CR1_PE);
```

Now the peripheral is ready for communication.

Complete Initialization Sequence

```
Enable RCC Clock

      |
      ▼
Configure GPIO

      |
      ▼
Disable PE

      |
      ▼
Configure CR1

      |
```

```
      ▼
Configure OAR1

      |
      ▼
Configure TIMINGR

      |
      ▼
Clear CR2

      |
      ▼
Enable PE
```

Registers Used

Register	Purpose
CR1	Enable peripheral and configure features
TIMINGR	Configure I²C timing
OAR1	Own slave address
CR2	Transfer configuration
ISR	Status flags (used later)
ICR	Clear interrupt flags (used later)

Important Code Snippets

Disable Peripheral

```
pI2Cx->CR1 &= ~(1 << I2C_CR1_PE);
```

Configure Own Address

```
tempreg = 0;

tempreg |= (OwnAddress << 1);

tempreg |= (1 << 15);

pI2Cx->OAR1 = tempreg;
```

Configure TIMINGR

```
pI2Cx->TIMINGR = timingValue;
```

Enable Peripheral

```
pI2Cx->CR1 |= (1 << I2C_CR1_PE);
```

Key Takeaways

- The STM32L452RE uses the second-generation I²C peripheral, which differs significantly from the legacy STM32F1/F4 implementation.
- The peripheral must be disabled (PE = 0) before modifying the TIMINGR register.
- TIMINGR replaces the legacy CCR and TRISE registers, combining all timing parameters into a single register.
- The CR1 register configures peripheral features such as analog filtering, clock stretching, interrupts, and DMA. ACK control is no longer configured in CR1.
- OAR1 stores the device's own address when operating in slave mode.
- CR2 is left in its default state during initialization because it is configured separately for each transfer.
- The I²C peripheral is enabled only after all configuration registers have been programmed.
- This initialization procedure is fully compatible with the STM32L452RE and forms the foundation for implementing blocking, interrupt-driven, and DMA-based I²C drivers.

LECTURE 2: Configuring the TIMINGR Register (STM32L452RE)

Objective

In this lecture, we configure the TIMINGR (Timing Register) of the STM32L452RE I²C peripheral. Unlike older STM32 families, the STM32L4 series does not use separate CCR and TRISE registers. Instead, a single register, TIMINGR, defines the complete timing of the I²C bus.

By the end of this lecture, you will understand:

- The purpose of the TIMINGR register.
- The function of each timing field.
- How the timing value is determined.
- How to configure TIMINGR for Standard Mode (100 kHz), Fast Mode (400 kHz), and Fast Mode Plus (1 MHz).
- Why the peripheral must be disabled before modifying TIMINGR.

Initialization Progress

```
I2C_Init()

|
├── Disable Peripheral ✔
├── Configure CR1 ✔
├── Configure OAR1 ✔
├── Configure TIMINGR ✔ ← This lecture
├── Configure CR2 ✔
└── Enable Peripheral ✔
```

Step 1 — Why TIMINGR?

The STM32L452RE generates the SCL clock using the TIMINGR register.

Unlike the legacy peripheral:

Legacy I²C Peripheral	STM32L452RE
CCR	TIMINGR
TRISE	TIMINGR
Separate calculations	Single timing value

All timing parameters are contained in one register.

TIMINGR Register Layout

```
31                                              0

+--------+---------+---------+---------+---------+
| PRESC | SCLDEL | SDADEL | SCLH | SCLL |
+--------+---------+---------+---------+---------+

31:28    27:24     23:20     15:8      7:0
```

TIMINGR Fields

Field	Bits	Purpose
PRESC	31:28	Clock prescaler
SCLDEL	27:24	Data setup delay
SDADEL	23:20	Data hold delay
SCLH	15:8	SCL high period
SCLL	7:0	SCL low period

PRESC (Clock Prescaler)

The PRESC field divides the peripheral clock before it is used by the I²C timing generator.

```
Peripheral Clock
       |
       ▼
    PRESC Divider
       |
       ▼
 Internal I²C Clock
```

Increasing PRESC increases the overall SCL period.

SCLDEL (Data Setup Time)

SCLDEL determines how long the SDA line must remain stable before the next rising edge of SCL.

```
SDA
```

```
_____________

        setup time

SCL
_____/_____
```

Correct setup timing ensures that the receiving device samples valid data.

SDADEL (Data Hold Time)

SDADEL defines how long SDA remains valid after the falling edge of SCL.

```
SCL
____\_____

SDA
___________

      hold time
```

This value ensures compliance with the I²C specification.

SCLH (SCL High Period)

SCLH specifies the duration of the HIGH portion of the SCL clock.

```
SCL

______/______\______
        SCLH
```

A larger value increases the HIGH time.

SCLL (SCL Low Period)

SCLL specifies the LOW duration of the SCL clock.

```
SCL

____\________/___

       SCLL
```

Normally SCLL is greater than SCLH because of the I²C timing requirements.

Step 2 — Choosing the Timing Value

Unlike the STM32F4 series, developers do not manually calculate CCR or TRISE.

Instead, the TIMINGR value depends on:

- Peripheral clock frequency (PCLK1)
- Desired I²C speed
- Rise time
- Fall time
- Analog filter
- Digital filter

ST recommends generating this value using:

- STM32CubeMX
- STM32CubeIDE
- STM32 I²C Timing Configuration Tool

Example Timing Values

Assuming:

Peripheral Clock = 80 MHz

Typical timing values are:

I²C Mode	Speed	Example TIMINGR
Standard Mode	100 kHz	0x10909CEC
Fast Mode	400 kHz	0x00C0216C
Fast Mode Plus	1 MHz	Device/application dependent

Note: These values depend on the peripheral clock, rise/fall times, and filter settings. Always regenerate TIMINGR if any of these parameters change.

Step 3 — Programming TIMINGR

Once the timing value has been selected:

```
pI2Cx->TIMINGR = timingValue;
```

Example

```
pI2Cx->TIMINGR = 0x10909CEC;
```

Why Must PE Be Disabled?

According to the STM32L452RE Reference Manual, the TIMINGR register can only be modified when the peripheral is disabled.

```
CR1

PE = 0

↓

Modify TIMINGR

↓

PE = 1
```

Attempting to modify TIMINGR while PE is set has no effect and may result in undefined behavior.

Complete Timing Configuration Flow

```
Peripheral Disabled

        |
        ▼
Choose Timing Value

        |
        ▼
Write TIMINGR

        |
        ▼
Verify Configuration

        |
        ▼
Enable Peripheral
```

Registers Used

Register	Purpose
CR1	Disable/Enable peripheral
TIMINGR	Configure I²C bus timing

Important Code Snippets

Disable Peripheral

```
pI2Cx->CR1 &= ~(1 << I2C_CR1_PE);
```

Configure TIMINGR

```
pI2Cx->TIMINGR = timingValue;
```

Example Timing

```
pI2Cx->TIMINGR = 0x10909CEC;
```

Enable Peripheral

```
pI2Cx->CR1 |= (1 << I2C_CR1_PE);
```

Timing Configuration Flow

```
Peripheral Clock

        |
        ▼
Determine Desired Speed

        |
        ▼
Generate TIMINGR Value

        |
        ▼
Disable Peripheral
```

```
        |
        ▼
Program TIMINGR

        |
        ▼
Enable Peripheral
```

Key Takeaways

- The TIMINGR register completely replaces the legacy CCR and TRISE registers used in older STM32 families.
- TIMINGR contains five fields: PRESC, SCLDEL, SDADEL, SCLH, and SCLL, which together define the I²C bus timing.
- The timing value depends on the peripheral clock frequency, desired I²C speed, rise/fall times, and filter configuration.
- ST recommends generating TIMINGR values using STM32CubeMX or the official I²C Timing Configuration Tool rather than calculating them manually.
- The I²C peripheral must be disabled (PE = 0) before writing to TIMINGR.
- After programming TIMINGR and completing the remaining configuration, the peripheral can be safely enabled (PE = 1).
- Proper TIMINGR configuration is essential for reliable I²C communication in Standard Mode (100 kHz), Fast Mode (400 kHz), and Fast Mode Plus (1 MHz).

LECTURE 3: STM32L452RE I²C Master Transmission Sequence (Theory)

Objective

Before writing any driver code, it is essential to understand how the STM32L452RE performs an I²C master transmission.

Unlike earlier STM32 families, the STM32L452RE uses the I²C version 2 peripheral, which relies on:

- CR2 to configure transfers
- TXDR for data transmission
- ISR for status flags
- ICR for clearing interrupt and error flags

The communication process is controlled through hardware-generated flags rather than the legacy EV5/EV6 event model.

Communication Scenario

The following example assumes:

- STM32L452RE acts as the Master
- External device acts as the Slave
- Master sends data to the slave

```
STM32L452RE (Master)
          |
          |  START
          |
          ▼
     Slave Address
          |
          ▼
      Data Bytes
          |
          ▼
         STOP
```

Registers Used During Transmission

Register	Purpose
CR2	Configure transfer
TXDR	Write transmit data
ISR	Monitor transfer status
ICR	Clear status/error flags

Complete Master Transmission Flow

```
Configure CR2
        |
        ▼
Generate START
        |
        ▼
Address Phase
        |
        ▼
TXIS = 1
        |
        ▼
Write First Byte
        |
        ▼
TXIS = 1
        |
        ▼
Write Remaining Bytes
        |
        ▼
Transfer Complete (TC)
        |
        ▼
Generate STOP
        |
        ▼
STOPF
        |
        ▼
Clear STOP Flag
```

Step 1 — Configure the Transfer

Before communication begins, the software configures the CR2 register.

CR2 contains:

Field	Purpose
SADD	Slave address
RD_WRN	Read/Write direction
NBYTES	Number of bytes
AUTOEND	Automatic STOP generation
START	Generate START
STOP	Generate STOP

Unlike older STM32 devices, the transfer parameters are configured before communication starts.

Step 2 — Generate the START Condition

Communication begins by setting the START bit in CR2.

```
Software

↓

CR2.START = 1

↓

Hardware Generates START

↓

Address Transmission Begins
```

The hardware automatically clears the START bit after it has generated the START condition.

Step 3 — Address Phase

Immediately after the START condition, the peripheral transmits:

7-bit Slave Address
+
R/W Bit

For transmission,

R/W = 0

The addressed slave either:

- Returns ACK
- Returns NACK

If NACK is received,

the hardware sets the NACKF flag.

Step 4 — TXIS Flag

When the slave acknowledges the address,

the hardware sets

TXIS = 1

TXIS means:

The transmit data register is empty and ready to accept a new byte.

Unlike the older peripheral,

there is no SB,

no ADDR,

and no TxE sequence.

TXIS directly indicates that software may write data.

Step 5 — Writing Data

Software writes one byte into TXDR.

```
Software

↓

Write TXDR

↓

Hardware Sends Byte

↓

Slave ACK

↓

TXIS = 1 Again
```

Each successful transmission produces another TXIS event.

Data Transmission Pipeline

```
Write Byte 1

↓

Shift Register

↓

Transmit

↓

ACK

↓

TXIS

↓

Write Byte 2

↓

Repeat
```

The process repeats until every byte has been transmitted.

Step 6 — Transfer Complete

After the final byte,

the hardware sets

TC = 1

TC means

Transfer Complete.

At this point,

all programmed bytes have been transmitted successfully.

Unlike the legacy BTF flag,

TC indicates that the transfer defined by NBYTES has completed.

Step 7 — Generate STOP

The software may either:

- Generate STOP
- Generate Repeated START

For a normal transmission,

STOP is generated.

```
Software

↓

CR2.STOP = 1

↓

Hardware Generates STOP

↓

STOPF Flag Set
```

Step 8 — Clear STOPF

Once the STOP condition has been transmitted,

the ISR register reports

STOPF = 1

Software clears this flag by writing to the ICR register.

```
STOPF

↓

Write STOPCF in ICR

↓

STOPF Cleared
```

Complete Master Transmission Sequence

```
Configure CR2
↓
START
↓
Address
↓
ACK
↓
TXIS
↓
Write Byte
↓
TXIS
↓
Write Next Byte
↓
Repeat
↓
TC
↓
STOP
↓
STOPF
↓
Clear STOPF
```

Important Status Flags

Flag	Register	Meaning
TXIS	ISR	TXDR is ready for new data
TC	ISR	Transfer complete
STOPF	ISR	STOP generated
NACKF	ISR	Slave did not acknowledge
BUSY	ISR	Bus busy
TXE	ISR	TXDR empty (status information)

Comparison with Older STM32 I²C Peripheral

Older STM32	STM32L452RE
SB	START bit in CR2
ADDR	Automatic handling
TxE	TXIS
BTF	TC
DR	TXDR
SR1	ISR
SR2	ISR/ICR
EV5/EV6	Not used

Registers Used

Register	Purpose
CR2	Configure transfer
TXDR	Write transmit bytes
ISR	Read transfer status
ICR	Clear transfer flags

Key Takeaways

- The STM32L452RE uses the I²C version 2 peripheral, which replaces the legacy event-based interface with a transfer-oriented design.
- Before communication begins, the CR2 register is configured with the slave address, transfer direction, number of bytes, and START/STOP control bits.
- The START condition is generated by setting the START bit in CR2; the hardware then automatically begins the address phase.
- After the slave acknowledges the address, the TXIS flag indicates that the TXDR register is ready to accept a data byte.
- Each transmitted byte generates another TXIS event until all bytes specified by NBYTES have been sent.
- When the programmed transfer is finished, the TC (Transfer Complete) flag is set, allowing the software to generate a STOP condition or a repeated START.
- After STOP is transmitted, the STOPF flag is set and must be cleared by writing to the ICR register.
- The legacy flags SB, ADDR, BTF, SR1, SR2, and the EV5/EV6 event model are not used on the STM32L452RE.

> *This theory provides the foundation for the next lecture, where we will implement a blocking I2C_MasterSendData() API using the STM32L452RE registers (CR2, TXDR, ISR, and ICR).*

LECTURE 4: Implementing I2C_MasterSendData() (Part 1 – API Creation & Transfer Configuration)

Objective

In this lecture, we begin implementing the blocking I2C_MasterSendData() API for the STM32L452RE. The focus is on:

- Creating the API prototype
- Understanding the required parameters
- Configuring the CR2 register
- Programming the slave address
- Selecting the transfer direction
- Configuring the number of bytes (NBYTES)
- Generating the START condition

The actual data transmission will be implemented in the next lecture.

Driver Development Progress

```
I2C_MasterSendData()

|
├── Configure CR2            ✔
├── Generate START           ✔
├── Wait for TXIS            → Next
├── Send Data                → Next
├── Wait for TC              → Next
├── Generate STOP            → Next
└── Clear STOPF              → Next
```

Step 1 – Create the API

Prototype (i2c_driver.h)

```
void I2C_MasterSendData(I2C_Handle_t *pI2CHandle,
                        uint8_t *pTxBuffer,
                        uint32_t Len,
                        uint16_t SlaveAddr);
```

Function Parameters

Parameter	Type	Purpose
pI2CHandle	I2C_Handle_t *	Pointer to I²C handle
pTxBuffer	uint8_t *	Data buffer to transmit
Len	uint32_t	Number of bytes
SlaveAddr	uint16_t	7-bit slave address

Why is SlaveAddr Required?

Unlike SPI,

I²C is an address-based protocol.

Before any data is transmitted,

the master must first transmit the slave address.

Therefore,

every master transfer API requires the destination address.

Step 2 – Create the Function

```
void I2C_MasterSendData(I2C_Handle_t *pI2CHandle,
                        uint8_t *pTxBuffer,
                        uint32_t Len,
                        uint16_t SlaveAddr)
{

}
```

High-Level Flow

```
Configure CR2

↓

Generate START

↓

Transmit Data
```

```
↓

Transfer Complete

↓

Generate STOP
```

Step 3 – Configure CR2

Unlike the older STM32 I²C peripheral,

the STM32L452RE stores all transfer information inside CR2.

CR2 contains

Field	Purpose
SADD	Slave Address
RD_WRN	Read / Write
NBYTES	Number of Bytes
RELOAD	Reload Mode
AUTOEND	Automatic STOP
START	Generate START
STOP	Generate STOP

Every transfer begins by programming CR2.

Step 4 – Clear Previous Configuration

Always clear the programmable transfer fields before configuring a new transaction.

```
uint32_t tempreg = pI2CHandle->pI2Cx->CR2;

tempreg &= ~(I2C_CR2_SADD_Msk |
             I2C_CR2_RD_WRN |
             I2C_CR2_NBYTES_Msk |
             I2C_CR2_RELOAD |
             I2C_CR2_AUTOEND |
             I2C_CR2_START |
             I2C_CR2_STOP);
```

This prevents leftover configuration from a previous transfer.

Step 5 – Configure the Slave Address

For 7-bit addressing,

the address occupies bits [7:1].

```
tempreg |= (SlaveAddr << 1);
```

Example

```
Slave Address

0x52

↓

Binary

1010010

↓

Shift Left

10100100
```

The hardware automatically appends the R/W bit.

Step 6 – Configure Transfer Direction

Since this API performs a Master Transmit,

the RD_WRN bit must remain cleared.

RD_WRN = 0

Meaning

```
Master

↓
```

```
Write

↓

Slave
```

For a receive API,

this bit will be set.

Step 7 – Configure NBYTES

The hardware must know

how many bytes will be transmitted.

The value is programmed into NBYTES.

```
tempreg |= (Len << I2C_CR2_NBYTES_Pos);
```

Example

```
Len = 5

↓

NBYTES = 5
```

Now the peripheral knows

exactly how many bytes belong to this transfer.

Why NBYTES?

Unlike the legacy peripheral,

the STM32L452RE automatically counts transmitted bytes.

```
NBYTES = 5

↓

Byte 1

↓

Byte 2

↓

Byte 3

↓

Byte 4

↓

Byte 5

↓

TC Flag Set
```

No manual byte counting is required inside the peripheral.

Step 8 – Select STOP Behaviour

For this blocking driver,

STOP will be generated manually.

Therefore

AUTOEND = 0

Manual STOP generation provides greater control and mirrors the behavior of many low-level drivers.

Step 9 – Write CR2

After configuring all required fields,

write the temporary value back to CR2.

```
pI2CHandle->pI2Cx->CR2 = tempreg;
```

Step 10 – Generate START

Communication begins by setting the START bit.

```
pI2CHandle->pI2Cx->CR2 |= I2C_CR2_START;
```

Hardware immediately

- Generates START
- Sends slave address
- Waits for ACK
- Begins the transfer

No additional software action is required during this stage.

Complete Transfer Setup Flow

```
Clear CR2 Fields

↓

Configure Slave Address

↓

Configure Direction

↓

Configure NBYTES

↓

Configure AUTOEND

↓

Write CR2

↓

Generate START

↓

Hardware Sends Address
```

Registers Used

Register	Purpose
CR2	Configure transfer parameters

Important Code Snippets

Configure Slave Address

```
tempreg |= (SlaveAddr << 1);
```

Configure NBYTES

```
tempreg |= (Len << I2C_CR2_NBYTES_Pos);
```

Write CR2

```
pI2CHandle->pI2Cx->CR2 = tempreg;
```

Generate START

```
pI2CHandle->pI2Cx->CR2 |= I2C_CR2_START;
```

Complete Flow

```
Application

↓

I2C_MasterSendData()

↓

Configure CR2

↓

START Generated

↓

Address Sent
```

```
↓

Wait for TXIS

↓

Transmit Data

↓

Transfer Complete

↓

STOP
```

Key Takeaways

- The STM32L452RE I²C peripheral uses the CR2 register to configure every transfer before communication begins.
- The I2C_MasterSendData() API accepts four parameters: the I²C handle, transmit buffer, number of bytes, and the 7-bit slave address.
- Before configuring a new transfer, the relevant fields in CR2 should be cleared to avoid carrying over settings from previous transactions.
- The SADD field stores the slave address, RD_WRN selects the transfer direction, and NBYTES specifies how many bytes will be transmitted.
- For this blocking implementation, AUTOEND is disabled so that the software controls when the STOP condition is generated.
- After programming CR2, the transfer begins by setting the START bit. The hardware automatically generates the START condition and transmits the slave address.
- At this point, the driver waits for the TXIS flag before sending the first data byte. This will be implemented in the next lecture.

LECTURE 5: Implementing I2C_MasterSendData() (Part 2 – Waiting for TXIS and Transmitting Data)

Objective

In this lecture, we continue implementing the I2C_MasterSendData() API for the STM32L452RE. The focus is on:

- Understanding the TXIS status flag.
- Implementing a generic I2C_GetFlagStatus() helper function.
- Waiting until the transmit register is ready.
- Transmitting all data bytes.
- Understanding how the hardware automatically manages the byte counter.

Current Progress

```
I2C_MasterSendData()

|
├── Configure CR2            ✔
├── Generate START           ✔
├── Wait for TXIS            ✔
├── Send Data                ✔
├── Wait for TC              → Next
├── Generate STOP            → Next
└── Clear STOPF              → Next
```

Step 1 – Understanding the TXIS Flag

After the START condition and address phase are completed successfully, the STM32L452RE hardware sets the TXIS flag.

TXIS stands for

Transmit Interrupt Status

Although its name contains Interrupt, this flag is also used in blocking (polling) drivers.

The TXIS flag indicates that:

- The slave has acknowledged the address.
- The transmit data register (TXDR) is empty.
- The peripheral is ready to accept the next data byte.

```
START

↓

Address Sent

↓

Slave ACK

↓

TXIS = 1

↓

Software Writes TXDR
```

Why Wait for TXIS?

Writing to TXDR before TXIS becomes set can overwrite data or violate the hardware transfer sequence.

Therefore, software must always wait until TXIS becomes set before writing each byte.

Step 2 – Define Status Flag Macros

To improve readability, define status flag macros corresponding to the ISR register.

Example:

```
#define I2C_FLAG_TXIS      I2C_ISR_TXIS
#define I2C_FLAG_TC        I2C_ISR_TC
#define I2C_FLAG_STOPF     I2C_ISR_STOPF
#define I2C_FLAG_BUSY      I2C_ISR_BUSY
#define I2C_FLAG_NACKF     I2C_ISR_NACKF
#define I2C_FLAG_TXE       I2C_ISR_TXE
```

Why Use I2C_FLAG_xxx?

Using a consistent naming convention provides several benefits:

- Improved readability
- Easier maintenance
- Better IDE auto-completion

- Consistent coding style across the driver

Example suggestions:

- I2C_FLAG_TXIS
- I2C_FLAG_TC
- I2C_FLAG_STOPF
- I2C_FLAG_BUSY
- I2C_FLAG_NACKF
- I2C_FLAG_TXE

Step 3 – Implement I2C_GetFlagStatus()

Instead of checking the ISR register directly throughout the driver, create a reusable helper function.

Prototype

```
uint8_t I2C_GetFlagStatus(I2C_RegDef_t *pI2Cx,
                          uint32_t FlagName);
```

Function Implementation

```
uint8_t I2C_GetFlagStatus(I2C_RegDef_t *pI2Cx,
                          uint32_t FlagName)
{
    if(pI2Cx->ISR & FlagName)
    {
        return SET;
    }

    return RESET;
}
```

Function Parameters

Parameter	Purpose
pI2Cx	Pointer to I²C peripheral
FlagName	Status flag to check

Register Used

Register	Purpose
ISR	Contains all status flags

Flow of I2C_GetFlagStatus()

```
Read ISR

↓

Mask Requested Flag

↓

Flag Set?

┌──────────────┐
│              │
Yes            No
│              │
▼              ▼
Return SET     Return RESET
```

Step 4 – Wait for TXIS

Before transmitting each byte,

wait until TXIS becomes set.

```
while(!I2C_GetFlagStatus(pI2CHandle->pI2Cx,
                         I2C_FLAG_TXIS));
```

Why Use Logical NOT (!) ?

The helper function returns

Return Value	Meaning
SET	Flag is set
RESET	Flag is clear

Therefore,

```
while(!I2C_GetFlagStatus(...));
```

means

```
Keep Waiting

↓

TXIS = 0

↓

TXIS = 1

↓

Exit Loop
```

This creates a blocking wait until the peripheral is ready for the next byte.

Step 5 – Send Data

Once TXIS becomes set,

write one byte into TXDR.

```
pI2CHandle->pI2Cx->TXDR = *pTxBuffer;
```

Immediately after writing,

advance the pointer.

```
pTxBuffer++;
```

Reduce the remaining length.

```
Len--;
```

Complete Transmission Loop

```
while(Len > 0)
{
    while(!I2C_GetFlagStatus(pI2CHandle->pI2Cx,
                             I2C_FLAG_TXIS));

    pI2CHandle->pI2Cx->TXDR = *pTxBuffer;

    pTxBuffer++;

    Len--;
}
```

Example

Suppose

Buffer

[0x11][0x22][0x33]

Len = 3

First Iteration

```
Wait TXIS

↓

Write 0x11

↓

Pointer++

↓

Len = 2
```

Second Iteration

```
Wait TXIS

↓

Write 0x22

↓

Pointer++

↓

Len = 1
```

Third Iteration

```
Wait TXIS

↓

Write 0x33

↓

Pointer++

↓

Len = 0
```

Loop exits.

Hardware Byte Counter

Unlike the legacy STM32 I²C peripheral,

the STM32L452RE automatically tracks the number of transmitted bytes.

```
NBYTES = 3

↓

Byte 1 Sent

↓

Byte 2 Sent

↓

Byte 3 Sent

↓

TC Flag Set
```

Software only decrements the local Len variable to know when all bytes have been written into TXDR.

Complete Flow

```
START

↓

Address Phase

↓

TXIS = 1

↓

Write TXDR

↓

TXIS = 1

↓

Write Next Byte

↓

Repeat Until Len = 0

↓

Wait for TC
```

Registers Used

Register	Purpose
TXDR	Write transmit data
ISR	Read TXIS and other flags

Important Code Snippets

Wait for TXIS

```
while(!I2C_GetFlagStatus(pI2CHandle->pI2Cx,
                         I2C_FLAG_TXIS));
```

Generic Flag Check

```
uint8_t I2C_GetFlagStatus(I2C_RegDef_t *pI2Cx,
                          uint32_t FlagName)
{
    if(pI2Cx->ISR & FlagName)
        return SET;

    return RESET;
}
```

Data Transmission Loop

```
while(Len > 0)
{
    while(!I2C_GetFlagStatus(pI2CHandle->pI2Cx,
                             I2C_FLAG_TXIS));

    pI2CHandle->pI2Cx->TXDR = *pTxBuffer;

    pTxBuffer++;

    Len--;
}
```

Key Takeaways

- The TXIS flag indicates that the TXDR register is ready to accept a new data byte.
- A reusable I2C_GetFlagStatus() helper function simplifies checking status flags in the ISR register.
- Before writing every byte, the driver waits until TXIS = 1, ensuring the transmit register is ready.
- Each transmitted byte is written to TXDR, after which the transmit buffer pointer is incremented and the remaining byte count is decremented.
- The STM32L452RE hardware automatically counts transmitted bytes using the NBYTES field programmed in CR2.
- When all bytes have been transferred according to NBYTES, the peripheral sets the TC (Transfer Complete) flag.
- The next lecture will complete the blocking transmission by waiting for TC, generating the STOP condition, clearing the STOPF flag, and finalizing the I2C_MasterSendData() API.

LECTURE 6: Implementing I2C_MasterSendData() (Part 3 – Transfer Complete, STOP Generation, and API Completion)

Objective

In this lecture, we complete the implementation of the blocking I2C_MasterSendData() API for the STM32L452RE. The following tasks are covered:

- Wait for the Transfer Complete (TC) flag.
- Generate the STOP condition.
- Wait for the STOPF flag.
- Clear the STOPF flag using the ICR register.
- Complete the blocking master transmit API.

Current API Progress

```
I2C_MasterSendData()

|
├── Configure CR2            ✔
├── Generate START           ✔
├── Wait for TXIS            ✔
├── Transmit Data            ✔
├── Wait for TC              ✔
├── Generate STOP            ✔
├── Wait for STOPF           ✔
├── Clear STOPF              ✔
└── Transmission Complete    ✔
```

Step 1 – Why Wait for TC?

After all bytes have been written into TXDR, communication is not yet complete.

The final byte may still be:

- Inside the transmit shift register.
- Being transmitted on the I²C bus.
- Waiting for the slave's ACK.

Therefore, software must wait until the hardware indicates that the programmed transfer has completely finished.

The STM32L452RE indicates this by setting the TC (Transfer Complete) flag.

Understanding the TC Flag

The TC flag is located in the ISR register.

It becomes set when:

- All bytes specified by NBYTES have been transmitted.
- The last byte has left the shift register.
- The slave has acknowledged the final byte.
- No automatic STOP has been generated (AUTOEND = 0).

```
NBYTES Reaches Zero

↓

Final Byte Sent

↓

Slave ACK

↓

TC = 1
```

Step 2 – Wait for TC

The driver waits until TC becomes set.

```
while(!I2C_GetFlagStatus(pI2CHandle->pI2Cx,
                         I2C_FLAG_TC));
```

Once TC becomes set,

the peripheral is ready for either:

- STOP generation
- Repeated START

Since this API performs a normal write transaction,

the next step is STOP generation.

Step 3 – Generate STOP

The STOP condition is generated by setting the STOP bit in the CR2 register.

```
pI2CHandle->pI2Cx->CR2 |= I2C_CR2_STOP;
```

Hardware Behaviour

Unlike software,

the hardware controls the exact timing of the STOP condition.

```
Software

↓

Set STOP Bit

↓

Hardware Generates STOP

↓

Communication Ends

↓

STOPF = 1
```

The STOP condition is placed on the bus only when it is safe to terminate communication.

Step 4 – Wait for STOPF

After the STOP condition has been generated,

the hardware sets

STOPF = 1

The driver waits for this indication.

```
while(!I2C_GetFlagStatus(pI2CHandle->pI2Cx,
                         I2C_FLAG_STOPF));
```

Why Wait for STOPF?

Without waiting,

the function could return before the STOP condition actually appears on the bus.

Waiting guarantees that

- Communication has fully terminated.
- The bus is released.
- Another transfer may safely begin.

Step 5 – Clear the STOP Flag

Unlike the older STM32 peripherals,

the STM32L452RE clears status flags using the Interrupt Clear Register (ICR).

To clear STOPF,

write the STOPCF bit.

```
pI2CHandle->pI2Cx->ICR |= I2C_ICR_STOPCF;
```

Why Use ICR?

The ISR register is read-only.

Status flags are cleared through the dedicated ICR register.

```
ISR

↓

STOPF = 1

↓

Write STOPCF

↓

STOPF Cleared
```

This mechanism prevents accidental modification of status bits.

Step 6 – Complete the Function

After clearing STOPF,

the transmission has successfully completed.

The API returns to the application.

```
Application

↓

I2C_MasterSendData()

↓

Transfer Finished

↓

Return
```

Complete Function Flow

```
Configure CR2

↓

Generate START

↓

Address Phase

↓

TXIS

↓

Transmit All Bytes

↓

TC

↓

Generate STOP

↓

STOPF

↓

Clear STOPF

↓

Return
```

Registers Used

Register	Purpose
CR2	Generate STOP
ISR	Monitor TC and STOPF
ICR	Clear STOPF

Helper Functions Used

Function	Purpose
I2C_GetFlagStatus()	Read status flags

Complete Code Sequence

```
/* Wait until transfer completes */
while(!I2C_GetFlagStatus(pI2CHandle->pI2Cx,
                         I2C_FLAG_TC));

/* Generate STOP */
pI2CHandle->pI2Cx->CR2 |= I2C_CR2_STOP;

/* Wait for STOP detection */
while(!I2C_GetFlagStatus(pI2CHandle->pI2Cx,
                         I2C_FLAG_STOPF));
```

```
/* Clear STOP flag */
pI2CHandle->pI2Cx->ICR |= I2C_ICR_STOPCF;
```

Complete Blocking Transmission Sequence

```
Initialize I²C

↓

Configure CR2

↓

START

↓

Address

↓

TXIS

↓

Write Byte

↓

Repeat Until Len = 0

↓

TC

↓

Generate STOP

↓

STOPF

↓

Clear STOPF

↓

Return to Application
```

Difference Between TC and STOPF

Flag	Meaning	Software Action
TXIS	TXDR is ready for another byte	Write next data byte
TC	All programmed bytes have been transmitted	Generate STOP or Repeated START
STOPF	STOP condition detected	Clear STOPF using ICR

Complete I2C_MasterSendData() Flow

```
Application
      |
      ▼
Configure CR2
      |
      ▼
Generate START
      |
      ▼
Address Sent
      |
      ▼
TXIS = 1
      |
      ▼
Write TXDR
      |
      ▼
Repeat Until All Bytes Sent
      |
      ▼
TC = 1
      |
      ▼
```

```
Generate STOP
      |
      ▼
STOPF = 1
      |
      ▼
Clear STOPF
      |
      ▼
Return
```

Key Takeaways

- After all data bytes have been written to TXDR, the driver waits for the TC (Transfer Complete) flag to ensure the entire transfer has finished.
- With AUTOEND disabled, the software is responsible for generating the STOP condition by setting the STOP bit in CR2.
- The hardware generates the STOP condition at the appropriate time and then sets the STOPF flag in the ISR register.
- The driver waits for STOPF to ensure the STOP condition has actually been transmitted before returning to the application.
- Unlike older STM32 I²C peripherals, the STM32L452RE clears status flags through the ICR (Interrupt Clear Register) rather than by reading status registers or writing to the status register itself.
- Writing STOPCF in ICR clears the STOPF flag and prepares the peripheral for the next transaction.
- At this point, the blocking I2C_MasterSendData() API is fully implemented for the STM32L452RE, completing the sequence: START → Address → Data → STOP.

Next Lecture: We will implement the blocking I2C_MasterReceiveData() API, covering receive transfers, the RXNE/RXDR data path, repeated START operations, and handling single-byte and multi-byte receptions on the STM32L452RE.

LECTURE 7: Implementing I2C_MasterReceiveData() (Part 1 – Blocking Master Receive)

Objective

In this lecture, we implement the blocking I2C_MasterReceiveData() API for the STM32L452RE. Unlike transmission, the master receives data from the slave through the RXDR register while monitoring the RXNE status flag.

By the end of this lecture, the driver will:

- Create the I2C_MasterReceiveData() API.
- Configure the transfer for Read mode.
- Generate the START condition.
- Receive data using RXNE.
- Wait for Transfer Complete (TC).
- Generate the STOP condition.
- Clear the STOP flag.

Current Driver Progress

```
I2C Driver

│
├── I2C_Init()                    ✔
├── I2C_MasterSendData()          ✔
├── I2C_MasterReceiveData()       ← Current Lecture
├── Interrupt APIs                → Next
├── DMA APIs                      → Later
└── Error Handling                → Later
```

Receive Operation Overview

Unlike Master Transmit, the data direction is reversed.

```
Slave

↓

Data Bytes

↓

STM32L452RE

↓

RXDR

↓

Application Buffer
```

API Prototype

```
void I2C_MasterReceiveData(I2C_Handle_t *pI2CHandle,
                           uint8_t *pRxBuffer,
                           uint32_t Len,
                           uint16_t SlaveAddr);
```

Function Parameters

Parameter	Purpose
pI2CHandle	Pointer to I²C handle
pRxBuffer	Receive buffer
Len	Number of bytes to receive
SlaveAddr	Slave device address

High-Level Receive Flow

```
Configure CR2

↓

Generate START

↓
```

```
Address Phase
↓
RXNE
↓
Read RXDR
↓
Repeat
↓
TC
↓
STOP
↓
STOPF
↓
Clear STOPF
```

Step 1 – Configure CR2

As with transmission,

every receive operation begins by configuring CR2.

The following fields must be programmed:

Field	Value
SADD	Slave Address
RD_WRN	1 (Read)
NBYTES	Number of Bytes
AUTOEND	0
START	Generated later

Clear Previous Transfer

```
uint32_t tempreg = pI2CHandle->pI2Cx->CR2;

tempreg &= ~(I2C_CR2_SADD_Msk |
             I2C_CR2_RD_WRN |
             I2C_CR2_NBYTES_Msk |
             I2C_CR2_RELOAD |
             I2C_CR2_AUTOEND |
             I2C_CR2_START |
             I2C_CR2_STOP);
```

Configure Slave Address

```
tempreg |= (SlaveAddr << 1);
```

Select Read Direction

Unlike transmission,

the RD_WRN bit must now be set.

```
tempreg |= I2C_CR2_RD_WRN;
```

Meaning

```
Master
↓
Read
↓
Slave
```

Configure Number of Bytes

```
tempreg |= (Len << I2C_CR2_NBYTES_Pos);
```

Write CR2

```
pI2CHandle->pI2Cx->CR2 = tempreg;
```

Generate START

```
pI2CHandle->pI2Cx->CR2 |= I2C_CR2_START;
```

Immediately,

the hardware

- Generates START
- Sends the slave address
- Sends the Read bit
- Waits for ACK

Understanding RXNE

After the slave transmits one byte,

the hardware places it into the RXDR register.

Then

RXNE = 1

RXNE means

Receive Data Register Not Empty

Receive Flow

```
Slave Sends Byte

↓

RXDR Updated

↓

RXNE = 1

↓

Software Reads RXDR

↓

RXNE Cleared
```

Wait for RXNE

Before reading every byte,

wait until RXNE becomes set.

```
while(!I2C_GetFlagStatus(pI2CHandle->pI2Cx,
                         I2C_FLAG_RXNE));
```

Read One Byte

```
*pRxBuffer = pI2CHandle->pI2Cx->RXDR;
```

Update Buffer

```
pRxBuffer++;
Len--;
```

Complete Receive Loop

```
while(Len > 0)
{
    while(!I2C_GetFlagStatus(pI2CHandle->pI2Cx,
                             I2C_FLAG_RXNE));

    *pRxBuffer = pI2CHandle->pI2Cx->RXDR;

    pRxBuffer++;

    Len--;
}
```

Wait for Transfer Complete

After all bytes have been received,

wait until

TC = 1

```
while(!I2C_GetFlagStatus(pI2CHandle->pI2Cx,
                         I2C_FLAG_TC));
```

Generate STOP

```
pI2CHandle->pI2Cx->CR2 |= I2C_CR2_STOP;
```

Wait for STOPF

```
while(!I2C_GetFlagStatus(pI2CHandle->pI2Cx,
                         I2C_FLAG_STOPF));
```

Clear STOPF

```
pI2CHandle->pI2Cx->ICR |= I2C_ICR_STOPCF;
```

Complete Receive Sequence

```
Configure CR2

↓

Generate START

↓

Address + Read

↓

RXNE

↓

Read RXDR

↓

Repeat Until Len = 0

↓

TC

↓

Generate STOP

↓

STOPF

↓

Clear STOPF

↓

Return
```

Registers Used

Register	Purpose
CR2	Configure receive transfer
RXDR	Receive data register
ISR	Monitor RXNE, TC, STOPF
ICR	Clear STOPF

Important Code Snippets

Wait for RXNE

```
while(!I2C_GetFlagStatus(pI2CHandle->pI2Cx,
                         I2C_FLAG_RXNE));
```

Read RXDR

```
*pRxBuffer = pI2CHandle->pI2Cx->RXDR;
```

Receive Loop

```
while(Len > 0)
{
    while(!I2C_GetFlagStatus(pI2CHandle->pI2Cx,
                             I2C_FLAG_RXNE));

    *pRxBuffer = pI2CHandle->pI2Cx->RXDR;

    pRxBuffer++;

    Len--;
}
```

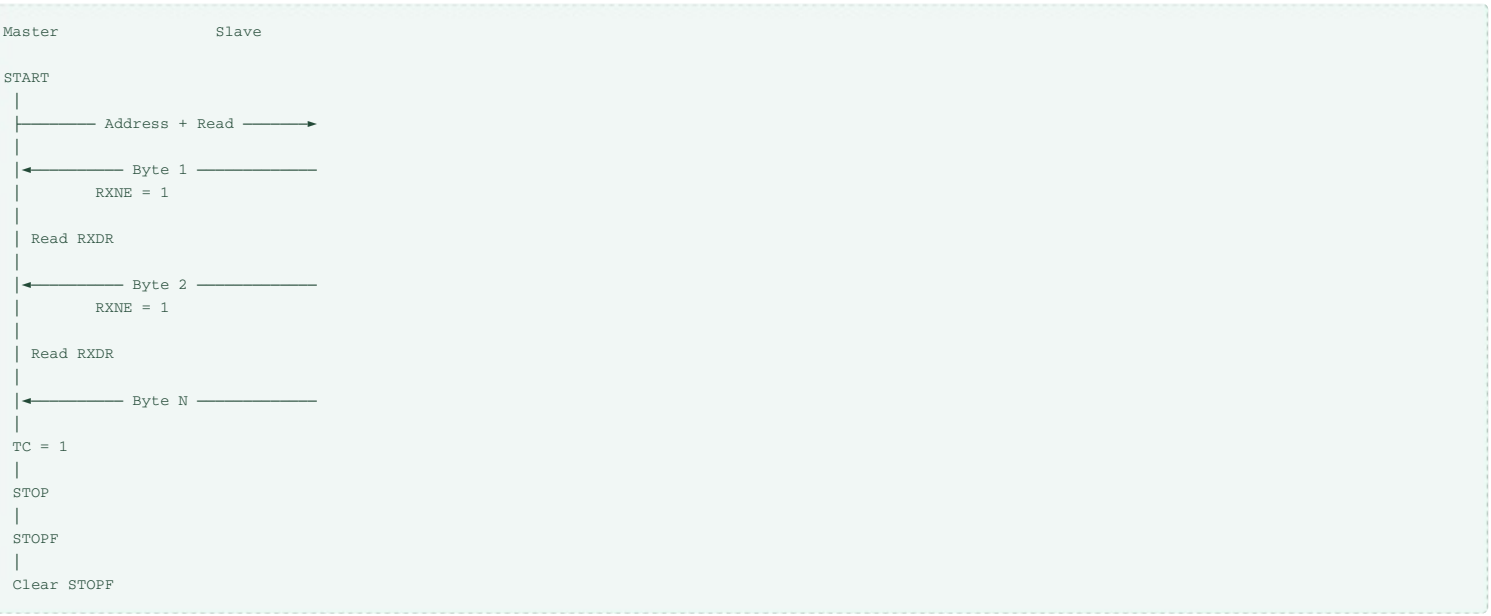
Generate STOP

```
pI2CHandle->pI2Cx->CR2 |= I2C_CR2_STOP;
```

Clear STOP Flag

```
pI2CHandle->pI2Cx->ICR |= I2C_ICR_STOPCF;
```

Receive Timing Diagram



Key Takeaways

- The I2C_MasterReceiveData() API uses the same transfer configuration mechanism as transmission but sets the RD_WRN bit to select Read mode.
- The hardware stores each received byte in the RXDR register and sets the RXNE (Receive Data Register Not Empty) flag.
- The driver waits for RXNE, reads the byte from RXDR, stores it in the application buffer, and repeats until all requested bytes have been received.
- After the programmed number of bytes has been received, the hardware sets the TC (Transfer Complete) flag.
- The software then generates a STOP condition, waits for the STOPF flag, and clears it using the ICR register.

LECTURE 8: I²C Error Handling in STM32L452RE (Blocking Driver)

Objective

In this lecture, we improve the robustness of the blocking I²C driver by handling common error conditions reported by the STM32L452RE I²C peripheral.

After completing this lecture, the driver will be able to detect:

- No Acknowledge (NACK)
- Bus Error
- Arbitration Lost
- Overrun/Underrun
- Timeout
- Busy Bus

Instead of remaining in an infinite loop, the driver will return an appropriate error code to the application.

Why Error Handling?

Until now, the driver waits for status flags using blocking loops such as:

```
while(!I2C_GetFlagStatus(pI2CHandle->pI2Cx, I2C_FLAG_TXIS));
```

If an error occurs, this loop may never exit.

Examples:

- Slave not connected
- Wrong slave address
- Bus held low
- Arbitration lost
- Hardware timeout

A robust driver should detect these conditions and return control to the application.

Error Flags in the ISR Register

The STM32L452RE reports errors through the ISR register.

Flag	Meaning
NACKF	Slave did not acknowledge
BERR	Bus error
ARLO	Arbitration lost
OVR	Overrun/Underrun
TIMEOUT	Timeout detected
BUSY	I²C bus busy

NACKF (No Acknowledge)

NACKF is set when:

- No slave exists at the specified address.
- The slave rejects a transmitted byte.

Example:

```
Master

↓

Address

↓

No ACK

↓

NACKF = 1
```

Detection:

```
if(pI2CHandle->pI2Cx->ISR & I2C_ISR_NACKF)
{
    /* Handle Error */
}
```

Clear the flag:

```
pI2CHandle->pI2Cx->ICR |= I2C_ICR_NACKCF;
```

BERR (Bus Error)

A Bus Error indicates an invalid condition on the I²C bus.

Possible causes:

- Corrupted START
- Corrupted STOP
- Electrical noise
- Clock glitches

Detection:

```
if(pI2CHandle->pI2Cx->ISR & I2C_ISR_BERR)
```

Clear:

```
pI2CHandle->pI2Cx->ICR |= I2C_ICR_BERRCF;
```

ARLO (Arbitration Lost)

Occurs only in multi-master systems.

Example:

```
Master 1

↓

Master 2

↓

Both Start Together

↓

Master Loses Arbitration

↓

ARLO = 1
```

Clear:

```
pI2CHandle->pI2Cx->ICR |= I2C_ICR_ARLOCF;
```

OVR (Overrun / Underrun)

Occurs when:

Receive:

Software fails to read RXDR before another byte arrives.

Transmit:

Software cannot supply data quickly enough.

Clear:

```
pI2CHandle->pI2Cx->ICR |= I2C_ICR_OVRCF;
```

TIMEOUT

Occurs when:

- SCL remains low too long.
- Communication stalls.
- Bus becomes locked.

Clear:

```
pI2CHandle->pI2Cx->ICR |= I2C_ICR_TIMOUTCF;
```

BUSY Flag

BUSY indicates

another transfer is already using the bus.

Before starting communication:

```
while(I2C_GetFlagStatus(pI2CHandle->pI2Cx,
                        I2C_FLAG_BUSY));
```

Once BUSY becomes RESET,

the driver may begin a new transfer.

Creating Driver Status Codes

Instead of returning void, return a status value.

```
typedef enum
```

```
{
    I2C_READY = 0,
    I2C_BUSY,
    I2C_ERROR_NACK,
    I2C_ERROR_BERR,
    I2C_ERROR_ARLO,
    I2C_ERROR_OVR,
    I2C_ERROR_TIMEOUT
}I2C_Status_t;
```

Updated API

Instead of

```
void I2C_MasterSendData(...);
```

use

```
I2C_Status_t I2C_MasterSendData(...);
```

Likewise,

```
I2C_Status_t I2C_MasterReceiveData(...);
```

Generic Error Check Function

```
static I2C_Status_t I2C_CheckErrors(I2C_RegDef_t *pI2Cx)
{
    if(pI2Cx->ISR & I2C_ISR_NACKF)
    {
        pI2Cx->ICR |= I2C_ICR_NACKCF;
        return I2C_ERROR_NACK;
    }

    if(pI2Cx->ISR & I2C_ISR_BERR)
    {
        pI2Cx->ICR |= I2C_ICR_BERRCF;
        return I2C_ERROR_BERR;
    }

    if(pI2Cx->ISR & I2C_ISR_ARLO)
    {
        pI2Cx->ICR |= I2C_ICR_ARLOCF;
        return I2C_ERROR_ARLO;
    }

    if(pI2Cx->ISR & I2C_ISR_OVR)
    {
        pI2Cx->ICR |= I2C_ICR_OVRCF;
        return I2C_ERROR_OVR;
    }

    if(pI2Cx->ISR & I2C_ISR_TIMEOUT)
    {
        pI2Cx->ICR |= I2C_ICR_TIMOUTCF;
        return I2C_ERROR_TIMEOUT;
    }

    return I2C_READY;
}
```

Using Error Checking

Instead of

```
while(!I2C_GetFlagStatus(...));
```

use

```
while(!I2C_GetFlagStatus(pI2CHandle->pI2Cx,
                         I2C_FLAG_TXIS))
{
    I2C_Status_t status =
        I2C_CheckErrors(pI2CHandle->pI2Cx);

    if(status != I2C_READY)
        return status;
}
```

Now,

the driver never blocks forever.

Complete Driver Flow

```
Wait BUSY = 0

↓

Configure CR2
```

```
↓
Generate START
↓
TXIS ?
↓
Yes
↓
Send Byte
↓
No
↓
Check Errors
↓
Return Error
↓
Continue Transfer
↓
TC
↓
STOP
↓
STOPF
↓
Return SUCCESS
```

Registers Used

Register	Purpose
ISR	Read status and error flags
ICR	Clear error flags
CR2	Generate START/STOP

Error Summary

Error	Flag	Clear Bit
No ACK	NACKF	NACKCF
Bus Error	BERR	BERRCF
Arbitration Lost	ARLO	ARLOCF
Overrun	OVR	OVRCF
Timeout	TIMEOUT	TIMOUTCF

Key Takeaways

- The STM32L452RE reports communication errors through dedicated flags in the ISR register.
- Each error flag is cleared by writing the corresponding clear bit in the ICR register.
- Returning an I2C_Status_t value instead of void allows the application to react to failures, such as retrying a transfer or reporting an error.
- A reusable I2C_CheckErrors() helper function centralizes error detection and simplifies the transmit and receive APIs.
- Integrating error checks into polling loops prevents the driver from hanging indefinitely if a device is absent, the bus is busy, or another communication fault occurs.
- With initialization, blocking transmit, blocking receive, and error handling implemented, the STM32L452RE blocking I²C driver is complete.

Complete Course Summary

You now have a complete STM32L452RE I²C Blocking Driver consisting of:

- ✅ I²C Peripheral Overview
- ✅ I2C_Init()
- ✅ TIMINGR Configuration
- ✅ Master Transmission Theory
- ✅ I2C_MasterSendData() (Blocking)
- ✅ I2C_MasterReceiveData() (Blocking)
- ✅ Error Handling

This forms a solid foundation for embedded development on the STM32L452RE. Interrupt-driven and DMA-based transfers can be added later without changing the basic driver architecture.